UNIVERSIDAD NACIONAL DE LOJA

INGENIERÍA EN SISTEMAS/CARRERA COMPUTACIÓN

Teoría de Autómatas y Computabilidad Avanzada

Evaluación Sumativa 1 · Unidad 1 · Informe técnico

Monitor de secuencias de eventos en el préstamo de libros de una biblioteca mediante autómatas finitos

Equipo: book

Autores:

Integrante	Rol
Jorge Luzuriaga	Responsable del modelo formal
Maria Chuico	Desarrollador del monitor
Mateo Chamba	Responsable de datos y pruebas
Emilio Jaramillo	Documentador

Docente:

Ing. Ing. Genoveva Suing

Septiembre 2026 – marzo 2027

Loja, Ecuador · octubre de 2026

1. Introducción

En los sistemas de gestión de las bibliotecas universitarias, cada movimiento de un libro queda registrado ya sea una reserva, un préstamo, una renovación, su devolución o incluso la generación y pago de una multa. Si vemos el historial de un usuario con un libro específico como una "sesión" y le asignamos un símbolo a cada uno de estos eventos, toda esa sesión se convierte en una cadena sobre un alfabeto. De esta forma las secuencias que nos interesa monitorear pasan a formar un lenguaje formal.

Analizar estas bitácoras es clave porque ciertas secuencias nos pueden mostrar si se están rompiendo las políticas de préstamo o si existen errores en el sistema. Por ejemplo, podemos detectar renovaciones abusivas, préstamos entregados a usuarios que acaban de recibir una multa, o devoluciones de libros que, según el sistema, nunca se llegaron a prestar. El problema que viene a resolver nuestro monitor es justamente detectar de forma automática este tipo de anomalías directamente en las bitácoras, evitándose el trabajo de revisar cada historial a mano. Todo esto se desarrolla siguiendo los lineamientos de la evaluación [6].

Con esto, el trabajo busca responder a la pregunta principal de la evaluación: ¿qué secuencias de eventos de un sistema real vale la pena vigilar, y cómo puede un autómata finito ayudarnos a detectarlas? Para lograrlo, modelamos tres patrones utilizando autómatas finitos no deterministas (AFND) [1]. Después, los conectamos mediante transiciones ε y los transformamos en su equivalente más simplificado: un autómata finito determinista (AFD) mínimo. Por último, implementamos un monitor en Python que ejecuta este AFD mínimo sobre nuestro propio conjunto de datos para así calcular las métricas de detección.

2. Modelo formal

2.1 Diccionario de eventos (alfabeto Σ)

Tabla 1. Diccionario de eventos del escenario de préstamo de libros.

Evento	Símbolo	Descripción	Relevancia
RESERVAR	R	Un usuario reserva el libro	Irrelevante (bucle)
PRESTAR	P	Se presta el libro a un usuario	Relevante (P2, P3)
RENOVAR	N	Se renueva el préstamo	Relevante (P1)
DEVOLVER	D	El usuario devuelve el libro	Relevante (P3)
MULTA	M	Se registra una multa al usuario	Relevante (P2)
PAGAR_MULTA	G	El usuario paga la multa	Irrelevante (bucle)

Σ = {R, P, N, D, M, G}. Los eventos R y G no forman parte de ningún patrón: los AFND los consumen únicamente con bucles y nunca producen una anomalía. Como ocurre con cualquier símbolo ajeno al patrón, si aparecen en medio de una secuencia de contigüidad (NNN, MP) la interrumpen, tal como lo establecen las definiciones de los lenguajes.

Ejemplos de sesión convertida en cadena. El historial «reservar, prestar, renovar, devolver» produce w = RPND (normal). El historial de la sesión S-0020 («prestar, reservar, renovar, reservar, multa, prestar, reservar, prestar, devolver») produce w = PRNRMPRPD (anómala por P2).

2.2 Patrones y lenguajes

Tabla 2. Patrones definidos para el monitor.

N.º	Descripción en lenguaje natural	Lenguaje (notación de conjuntos)	Tipo	¿AFND?
P1	Tres renovaciones seguidas (posible abuso de la política)	$L_1 = \{w \in \Sigma^* \mid w \text{ contiene NNN}\}$	Anomalía	Sí
P2	Préstamo inmediatamente después de una multa	$L_2 = \{w \in \Sigma^* \mid w \text{ contiene MP}\}$	Anomalía	Sí
P3	Devolución de un libro que nunca se prestó en la sesión	$L_3 = \{w \in \Sigma^* \mid w \text{ contiene D y no hay P antes de la primera D}\}$	Anomalía	No (diseño natural determinista)

Los patrones P1 y P2 son del tipo «la sesión contiene la secuencia…», cuyo diseño natural es un AFND. Todos tienen una longitud máxima de 4 símbolos (NNN: 3; MP: 2; P3 involucra solo a P y D). Pertenencia de cadenas de ejemplo a cada lenguaje (comprobada con los autómatas construidos):

Tabla 3. Cadenas que pertenecen y que no pertenecen a cada lenguaje.

Patrón	Pertenecen ($w \in L_i$)	No pertenecen ($w \notin L_i$) y motivo
P1	PNNN, RPNNNDG	PNNDN (solo dos N seguidas); PNRNN (una R interrumpe la racha)
P2	DMP, PMP	MGP (el pago G se interpone entre M y P); PMDGP (D y G se interponen)
P3	DPN, RD	PND (hay P antes de la primera D); RPD (hay P antes de la primera D)

3. Conjunto de datos

En este trabajo utilizamos datos sintéticos generados mediante el script datos/generador.py. Para asegurar que los resultados sean completamente reproducibles, configuramos una semilla aleatoria fija (2026). Cada sesión se construye simulando el ciclo de vida natural de un préstamo (pasando de libre a prestado, y llegando a una posible multa), mientras que las sesiones anómalas se crean inyectando intencionalmente el patrón de error correspondiente.

Es importante destacar que la etiqueta de cada sesión no es asignada por los autómatas. En su lugar, calcula un "oráculo" independiente que evalúa la sesión aplicando al pie de la letra las definiciones de los lenguajes L_1, L_2 y L_3.

Además de los datos sintéticos, se generó una versión híbrida (datos/datos_real.py) que toma 50 000 préstamos reales de la Seattle Public Library [4] y construye alrededor de ellos los demás eventos (renovar, devolver, multa y pago de multa) de forma sintética, con semilla fija. Solo los préstamos (P) son reales: 408 de 1 818 eventos en validación (22,4 %) y 62 de 278 en prueba (22,3 %). Cada sesión corresponde a un ejemplar distinto (400 y 60), tiene de 3 a 11 eventos y las anomalías son inyectadas, por lo que las métricas miden la correcta construcción de los autómatas y no el comportamiento real de una biblioteca. Los préstamos reales incluyen libros (56 %), DVD, CD y kits.

Tabla 4. Columnas del archivo CSV (un renglón por evento).

Columna	Ejemplo	Uso
id_evento	1045	Orden y trazabilidad
fecha_hora	2026-09-26 16:10:00	Ordena los eventos de la sesión (el autómata no usa el tiempo)
id_sesion	S-0020	Agrupa los eventos: cada sesión es una cadena
actor	U-D26F	Usuario anonimizado (código hexadecimal aleatorio)
evento / simbolo	PRESTAR / P	Nombre del evento y símbolo del alfabeto
etiqueta	normal / anómala	Solo en el archivo de validación y en la clave del archivo de prueba
tipo_patron	P2 (o «ninguno»)	Patrón que explica la etiqueta
casi_anomala	sí / no	Columna adicional: sesión normal cercana a un patrón (p. ej., dos N seguidas)

Tabla 5. Características de los archivos generados.

Característica	validacion.csv	prueba.csv
Sesiones	400	60
Eventos	3 724	560
Eventos por sesión (mín.–máx.)	3–19 (media 9,31)	3–17
Sesiones anómalas	72 (18,0 %): 24 por patrón	11 (18,3 %)
Sesiones casi anómalas (normales)	166	incluidas
Etiquetas	en el mismo archivo	sin etiquetas; clave_prueba.csv por separado

Las sesiones casi anómalas son normales pero cercanas a un patrón: contienen NN sin llegar a NNN, o una M seguida de P pero separadas por uno o dos eventos (por ejemplo, MDP). Los eventos irrelevantes R y G aparecen a lo largo de todo el archivo. Fragmento de la sesión S-0020:

Tabla 6. Fragmento de validacion.csv (sesión S-0020).

id_evento	fecha_hora	id_sesion	actor	evento	simbolo	etiqueta	tipo_patron	casi_anomala
199	2026-09-23 16:07:00	S-0020	U-D26F	RESERVAR	R	anómala	P2	no
200	2026-09-25 18:12:00	S-0020	U-D26F	MULTA	M	anómala	P2	no
201	2026-09-26 16:10:00	S-0020	U-D26F	PRESTAR	P	anómala	P2	no
202	2026-09-29 15:49:00	S-0020	U-D26F	RESERVAR	R	anómala	P2	no
203	2026-10-01 18:27:00	S-0020	U-D26F	PRESTAR	P	anómala	P2	no
204	2026-10-04 09:56:00	S-0020	U-D26F	DEVOLVER	D	anómala	P2	no

4. Diseño de los AFND

Cada patrón tiene su AFND $M_i = (Q_i, \Sigma, \delta_i, q_0, F_i)$ con Σ = {R, P, N, D, M, G}. En las tablas, => marca el estado inicial y * los estados de aceptación; «—» indica que no hay transición ($\delta(q, a) = \varnothing$) y las llaves, un conjunto de destinos.

4.1 AFND del patrón P1

Quíntuple. Q_1 = {A0, A1, A2, A3}, q_0 = A0, F_1 = {A3}; δ_1 según la Tabla 7. A0: aún no hay renovaciones seguidas (cualquier otro evento lo mantiene); A1: se vio una N; A2: dos N seguidas; A3: tres N seguidas, patrón detectado (estado absorbente).

Tabla 7. Tabla de transiciones del AFND de P1 (4 estados, 15 transiciones).

δ	R	P	N	D	M	G
→ A0	A0	A0	{A0,A1}	A0	A0	A0
A1	—	—	A2	—	—	—
A2	—	—	A3	—	—	—
A3 *	A3	A3	A3	A3	A3	A3

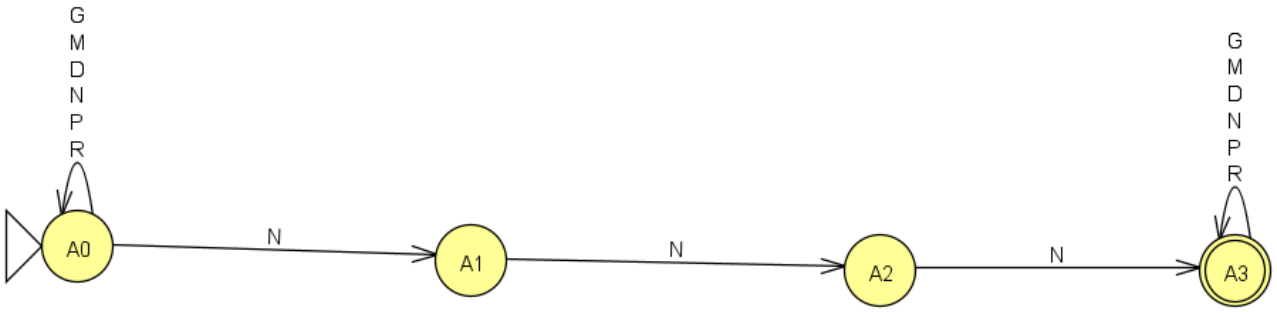


Figura 1. Diagrama del AFND de P1 (JFLAP: automatas/afnd_p1.jff).

Es no determinista porque, estando en A0, el símbolo N lleva a dos destinos, {A0, A1}: el autómata «adivina» en qué N empieza la racha de tres.

4.2 AFND del patrón P2

Quíntuple. Q_2 = {B0, B1, B2}, q_0 = B0, F_2 = {B2}; δ_2 según la Tabla 8. B0: no hay una multa inmediatamente anterior; B1: el último evento fue una multa (M); B2: una P siguió a la M, patrón detectado (absorbente).

Tabla 8. Tabla de transiciones del AFND de P2 (3 estados, 14 transiciones).

δ	R	P	N	D	M	G
→ B0	B0	B0	B0	B0	{B0,B1}	B0
B1	—	B2	—	—	—	—
B2 *	B2	B2	B2	B2	B2	B2

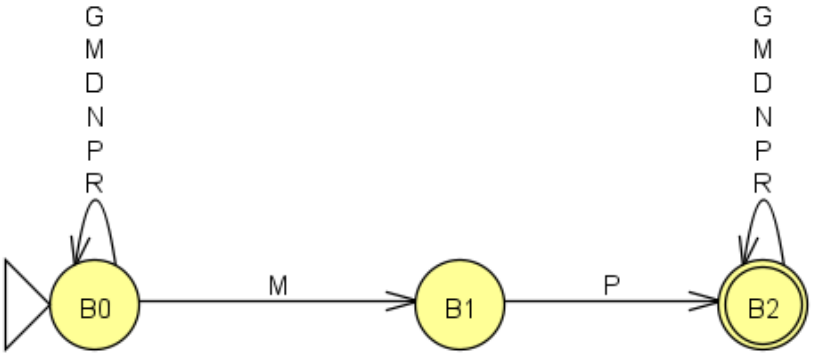


Figura 2. Diagrama del AFND de P2 (JFLAP: automatas/afnd_p2.jff).

Es no determinista porque, en B0, el símbolo M lleva a {B0, B1}: el autómata «adivina» cuál M es la que precede al préstamo.

4.3 AFND del patrón P3

Quíntuple. Q_3 = {C0, C1}, q_0 = C0, F_3 = {C1}; δ_3 según la Tabla 9. C0: todavía no ha ocurrido ni P ni D (solo R, N, M o G); C1: llegó una D antes que cualquier P, patrón detectado (absorbente).

Tabla 9. Tabla de transiciones del AFND de P3 (2 estados, 11 transiciones).

δ	R	P	N	D	M	G
→ C0	C0	—	C0	C1	C0	C0
C1 *	C1	C1	C1	C1	C1	C1

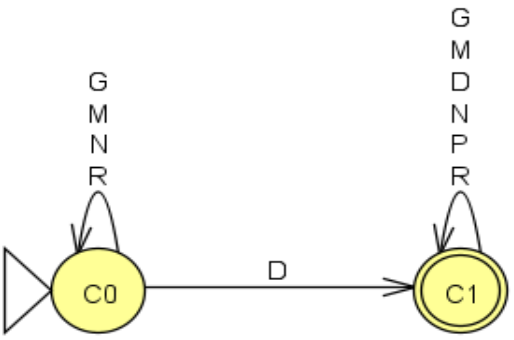


Figura 3. Diagrama del AFND de P3 (JFLAP: automatas/afnd_p3.jff).

No presenta indeterminismo: cada par (estado, símbolo) tiene a lo sumo un destino y δ es parcial (en C0 no hay transición con P: si se presta antes de devolver, la ejecución se bloquea y el patrón queda descartado). Por eso su diseño natural es determinista y se modela como AFND solo para unirlo con los demás.

5. Unión con transiciones ε y su eliminación

Los tres AFND se unen en un solo autómata [1] con un nuevo estado inicial s y tres transiciones ε hacia los estados iniciales de cada patrón (δ(s, ε) = {A0, B0, C0}). El AFND unido tiene 10 estados y 43 transiciones (3 de ellas ε) y acepta la unión $L_1 \cup L_2 \cup L_3$ con F = {A3, B2, C1}.

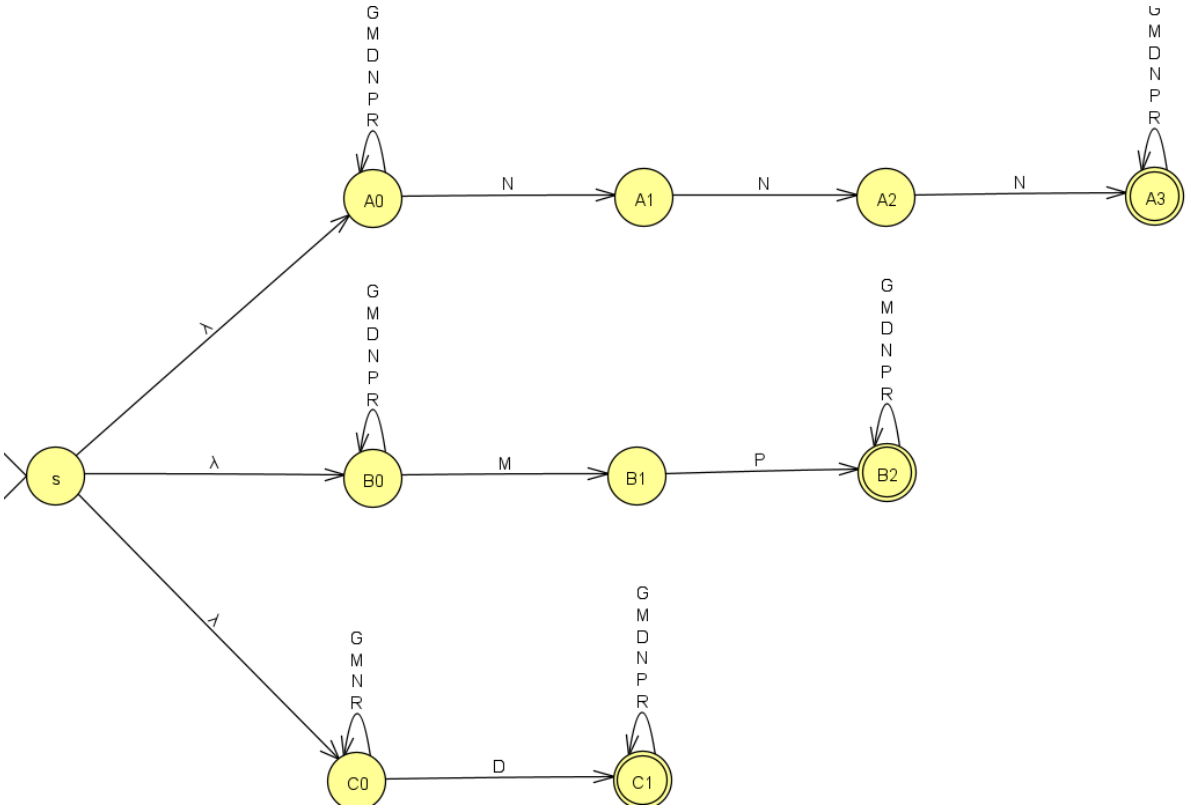


Figura 4. AFND unido con transiciones ε (automatas/afnd_union_eps.jff).

Eliminación de ε, paso a paso:

1. Se calcula la ε-clausura de cada estado: ε-clausura(s) = {A0, B0, C0, s}; para los demás estados, ε-clausura(q) = {q}.
2. Se define δ'(q, a) = ε-clausura(δ(ε-clausura(q), a)). Por ejemplo, δ'(s, N) = ε-clausura(δ(s, N) ∪ δ(A0, N) ∪ δ(B0, N) ∪ δ(C0, N)) = {A0, A1, B0, C0}.
3. Los estados de aceptación son F' = {q | ε-clausura(q) ∩ F ≠ ∅} = {A3, B2, C1}: ninguna clausura nueva contiene un estado final, así que s no es de aceptación.
4. Se eliminan las transiciones ε y los estados inaccesibles (ninguno). El resultado tiene 10 estados y 59 transiciones (Tabla 10).

Tabla 10. δ' del AFND sin transiciones ε (automatas/afnd_sin_eps.jff).

δ'	R	P	N	D	M	G
→ s	{A0,B0,C0}	{A0,B0}	{A0,A1,B0,C0}	{A0,B0,C1}	{A0,B0,B1,C0}	{A0,B0,C0}
A0	A0	A0	{A0,A1}	A0	A0	A0
A1	—	—	A2	—	—	—
A2	—	—	A3	—	—	—
A3 *	A3	A3	A3	A3	A3	A3
B0	B0	B0	B0	B0	{B0,B1}	B0
B1	—	B2	—	—	—	—
B2 *	B2	B2	B2	B2	B2	B2
C0	C0	—	C0	C1	C0	C0
C1 *	C1	C1	C1	C1	C1	C1

6. Conversión a AFD y minimización

6.1 Construcción de subconjuntos

Se aplica la construcción de subconjuntos [1], [2] al AFND sin ε partiendo de {s} y explorando solo los estados accesibles. El AFD obtenido tiene 41 estados y 246 transiciones (completo: 41 × 6); 32 de ellos son de aceptación (los marcados con *).

Tabla 11. Tabla de la construcción de subconjuntos (estados accesibles).

Estado	Conjunto del AFND	R	P	N	D	M	G
→ q0	s	q1	q2	q3	q4	q5	q1
q1	{A0,B0,C0}	q1	q2	q3	q4	q5	q1
q2	{A0,B0}	q2	q2	q6	q2	q7	q2
q3	{A0,A1,B0,C0}	q1	q2	q8	q4	q5	q1
q4 *	{A0,B0,C1}	q4	q4	q9	q4	q10	q4
q5	{A0,B0,B1,C0}	q1	q11	q3	q4	q5	q1
q6	{A0,A1,B0}	q2	q2	q12	q2	q7	q2
q7	{A0,B0,B1}	q2	q11	q6	q2	q7	q2
q8	{A0,A1,A2,B0,C0}	q1	q2	q13	q4	q5	q1
q9 *	{A0,A1,B0,C1}	q4	q4	q14	q4	q10	q4
q10 *	{A0,B0,B1,C1}	q4	q15	q9	q4	q10	q4
q11 *	{A0,B0,B2}	q11	q11	q16	q11	q17	q11
q12	{A0,A1,A2,B0}	q2	q2	q18	q2	q7	q2
q13 *	{A0,A1,A2,A3,B0,C0}	q19	q20	q13	q21	q22	q19
q14 *	{A0,A1,A2,B0,C1}	q4	q4	q23	q4	q10	q4
q15 *	{A0,B0,B2,C1}	q15	q15	q24	q15	q25	q15
q16 *	{A0,A1,B0,B2}	q11	q11	q26	q11	q17	q11
q17 *	{A0,B0,B1,B2}	q11	q11	q16	q11	q17	q11
q18 *	{A0,A1,A2,A3,B0}	q20	q20	q18	q20	q27	q20
q19 *	{A0,A3,B0,C0}	q19	q20	q28	q21	q22	q19
q20 *	{A0,A3,B0}	q20	q20	q29	q20	q27	q20
q21 *	{A0,A3,B0,C1}	q21	q21	q30	q21	q31	q21
q22 *	{A0,A3,B0,B1,C0}	q19	q32	q28	q21	q22	q19
q23 *	{A0,A1,A2,A3,B0,C1}	q21	q21	q23	q21	q31	q21
q24 *	{A0,A1,B0,B2,C1}	q15	q15	q33	q15	q25	q15
q25 *	{A0,B0,B1,B2,C1}	q15	q15	q24	q15	q25	q15
q26 *	{A0,A1,A2,B0,B2}	q11	q11	q34	q11	q17	q11
q27 *	{A0,A3,B0,B1}	q20	q32	q29	q20	q27	q20
q28 *	{A0,A1,A3,B0,C0}	q19	q20	q13	q21	q22	q19
q29 *	{A0,A1,A3,B0}	q20	q20	q18	q20	q27	q20
q30 *	{A0,A1,A3,B0,C1}	q21	q21	q23	q21	q31	q21
q31 *	{A0,A3,B0,B1,C1}	q21	q35	q30	q21	q31	q21
q32 *	{A0,A3,B0,B2}	q32	q32	q36	q32	q37	q32
q33 *	{A0,A1,A2,B0,B2,C1}	q15	q15	q38	q15	q25	q15
q34 *	{A0,A1,A2,A3,B0,B2}	q32	q32	q34	q32	q37	q32
q35 *	{A0,A3,B0,B2,C1}	q35	q35	q39	q35	q40	q35
q36 *	{A0,A1,A3,B0,B2}	q32	q32	q34	q32	q37	q32
q37 *	{A0,A3,B0,B1,B2}	q32	q32	q36	q32	q37	q32
q38 *	{A0,A1,A2,A3,B0,B2,C1}	q35	q35	q38	q35	q40	q35
q39 *	{A0,A1,A3,B0,B2,C1}	q35	q35	q38	q35	q40	q35
q40 *	{A0,A3,B0,B1,B2,C1}	q35	q35	q39	q35	q40	q35

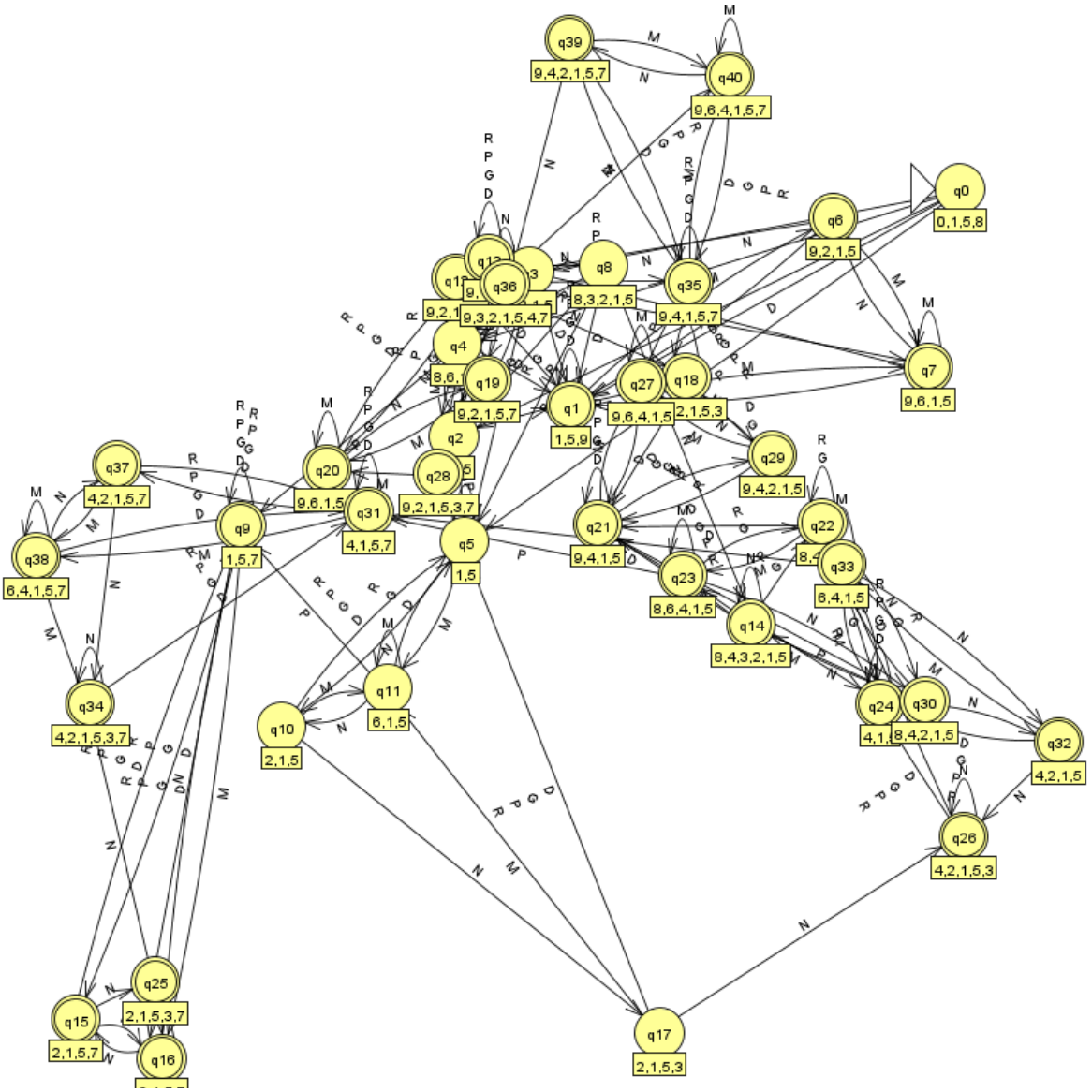


Figura 5. Conversión a AFD.

6.2 Minimización

Se aplica la partición de estados equivalentes (algoritmo de Moore). Iteración 0: dos bloques, no finales y finales (9 y 32 estados). Iteración 1: 7 bloques. Iteración 2: 9 bloques, partición estable. Los 32 subconjuntos de aceptación son equivalentes entre sí (todos absorben cualquier símbolo) y q0 (el estado {s}) es equivalente a q1, pues ambos se comportan igual ante cualquier cadena.

Tabla 12. Estados equivalentes (clases de la partición final).

Clase	Estados del AFD agrupados	Qué recuerda el estado
m0	q0, q1	sin N seguidas; sin M pendiente; sin P ni D
m1	q2	sin N seguidas; sin M pendiente; ya hubo P
m2	q3	1 N seguida(s); sin M pendiente; sin P ni D
m3 *	32 estados: q4, q9–q11, q13–q40	Anomalía detectada
m4	q5	sin N seguidas; última M; sin P ni D
m5	q6	1 N seguida(s); sin M pendiente; ya hubo P
m6	q7	sin N seguidas; última M; ya hubo P
m7	q8	2 N seguida(s); sin M pendiente; sin P ni D
m8	q12	2 N seguida(s); sin M pendiente; ya hubo P

El AFD mínimo tiene 9 estados y 54 transiciones: 78 % menos estados y 78 % menos transiciones que el AFD por subconjuntos.

Tabla 13. Tabla de transiciones del AFD mínimo (automatas/afd_minimo.jff).

δ	R	P	N	D	M	G
→ m0	m0	m1	m2	m3	m4	m0
m1	m1	m1	m5	m1	m6	m1
m2	m0	m1	m7	m3	m4	m0
m3 *	m3	m3	m3	m3	m3	m3
m4	m0	m3	m2	m3	m4	m0
m5	m1	m1	m8	m1	m6	m1
m6	m1	m3	m5	m1	m6	m1
m7	m0	m1	m3	m3	m4	m0
m8	m1	m1	m3	m1	m6	m1

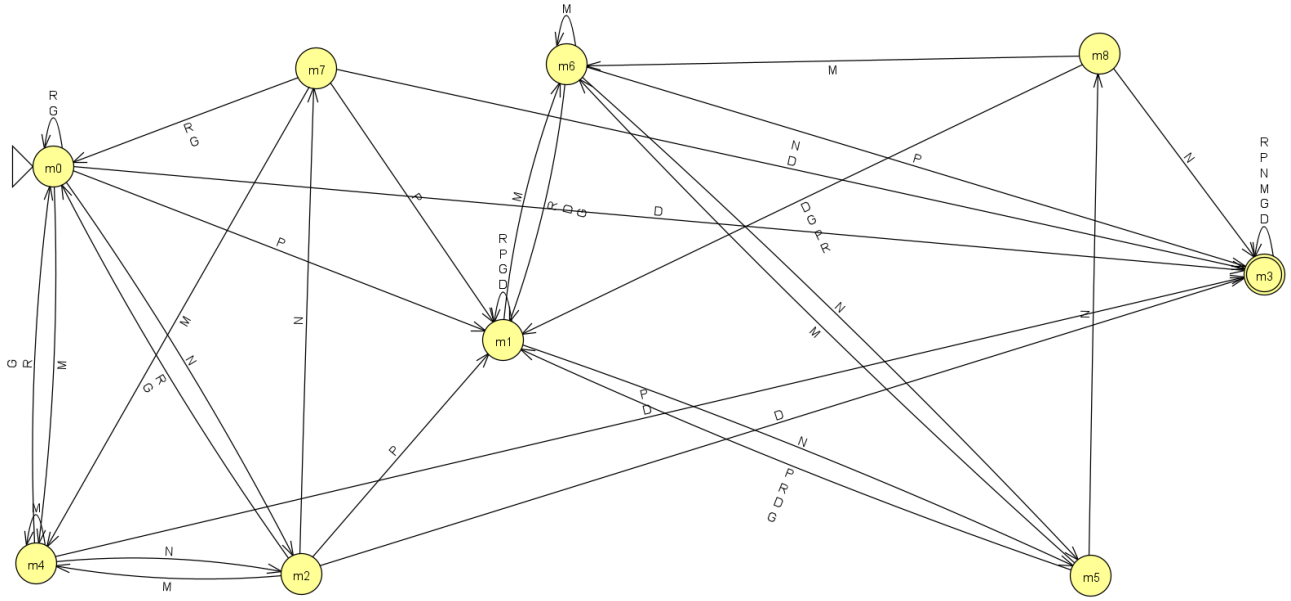


Figura 6. AFD mínimo (m3 es el estado de anomalía detectada; Σ − {N, M} = {R, P, D, G}). Es completo: cada estado tiene una transición por símbolo.

6.3 Verificación de equivalencia

Para verificar la equivalencia de los modelos, usamos la rutina equivalentes() de monitor/automata.py, que aplica el mismo criterio que la herramienta "Compare Equivalence" de JFLAP [3] y tambíen lo hicimos en el propio JFLAP.

Los resultados confirmaron que el AFD mínimo equivale al AFND unido con transiciones ε (tras explorar 41 pares), y que el AFD por subconjuntos equivale al AFND sin ε.

Para mayor seguridad, hicimos una prueba exhaustiva con 335 923 cadenas (secuencias de hasta 7 eventos) frente al oráculo, encontrando cero discrepancias en todos los modelos.

Tabla 14. Trazas sobre el AFD mínimo (para verificar a mano).

Cadena	Recorrido de estados	Resultado
RPND	m0 → m0 → m1 → m5 → m1	Normal (rechaza)
PNNDMGP	m0 → m1 → m5 → m8 → m1 → m6 → m1 → m1	Normal (rechaza)
PMP	m0 → m1 → m6 → m3	Anómala (acepta)
PNNNR	m0 → m1 → m5 → m8 → m3 → m3	Anómala (acepta)

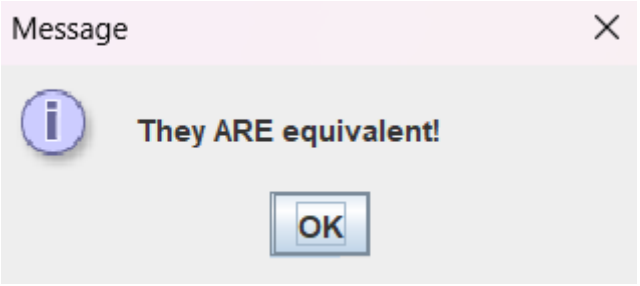


Figura 7. JFLAP, Compare Equivalence entre el AFD mínimo y el AFND unido.

7. Implementación del monitor

El código sigue la estructura pedida: `datos/generador.py` (genera los CSV), `monitor/automata.py` (simulación, unión con ε, eliminación de ε, subconjuntos, minimización y equivalencia), `monitor/construir.py` (construye y exporta los autómatas y calcula las métricas del informe) y `monitor/monitor.py` (lectura del CSV, detección y métricas). Solo se usa la biblioteca estándar de Python.

Representación: Modelamos los autómatas utilizando diccionarios. En el AFND, mapeamos un par de (estado, símbolo) hacia un conjunto de estados (usando el símbolo vacío para ε); en el AFD, el mapeo va hacia un único estado. Estos modelos se guardan en formato .json (el monitor carga específicamente afd_minimo.json) y en .jff para visualizarlos en JFLAP.

Procesamiento: El monitor lee el archivo CSV, agrupa y ordena cronológicamente los eventos por sesión, y une sus símbolos para formar una cadena de texto. Esta cadena se evalúa en el AFD mínimo. Si el sistema detecta que la sesión es anómala, la pasa también por los AFND (P1, P2 y P3) para identificar exactamente qué patrón causó el error. Finalmente, comparando los resultados con las etiquetas reales, el monitor calcula los verdaderos positivos (VP), falsos positivos (FP), falsos negativos (FN), la precisión y la exhaustividad.

```
def leer_sesiones(ruta):                          # monitor.py
    ...
    for sid, evs in eventos.items():
        evs.sort(key=lambda e: (e["fecha_hora"], int(e["id_evento"])))
        cadena = "".join(e["simbolo"] for e in evs)    # sesión -> cadena w

def acepta(self, w):                               # automata.py, clase AFD
    q = self.q0
    for a in w:
        q = self.delta[(q, a)]                     # una consulta a δ por símbolo
    return q in self.F

prediccion = {sid: afd.acepta(c) for sid, (c, _, _) in sesiones.items()}
```

Ejecución: `python datos/generador.py`, `python monitor/construir.py` y `python monitor/monitor.py --datos datos/validacion.csv` (para el archivo de prueba se agrega `--clave datos/clave_prueba.csv`).

Link de repositorio: https://github.com/mariaechm/Prestamo-de-libros-en-la-biblioteca---Automatas-U1-.git

8. Resultados

8.1 Métricas de detección

Tabla 15. Métricas del AFD mínimo (VP, FP, FN, VN y razones).

Archivo	VP	FP	FN	VN	Precisión	Exhaustividad
Validación (400 sesiones)	72	0	0	328	1,00	1,00
Prueba (60 sesiones)	11	0	0	49	1,00	1,00
Validación híbrida (400 sesiones)	72	0	0	328	1,00	1,00
Prueba híbrida (60 sesiones)	11	0	0	49	1,00	1,00

Tabla 16. Métricas por patrón sobre el archivo de validación (AFND de cada patrón).

Patrón	VP	FP	FN	VN	Precisión	Exhaustividad
P1	24	0	0	376	1,00	1,00
P2	24	0	0	376	1,00	1,00
P3	24	0	0	376	1,00	1,00

8.2 Comparación AFND – AFD

Tabla 17. Tamaño y tiempo de ejecución (400 sesiones; mejor de 40 repeticiones con time.perf_counter).

Autómata	Estados	Transiciones	Tiempo (µs por sesión)
AFND P1 / P2 / P3 (por separado)	4 / 3 / 2	15 / 14 / 11	15,18 (los tres)
AFND unido con ε	10	43	11,16
AFND sin ε	10	59	10,18
AFD por subconjuntos	41	246	0,66
AFD mínimo	**9**	**54**	**0,66**

El AFD mínimo procesa una sesión en 0,66 µs frente a 11,16 µs del AFND unido con ε (unas 17 veces más rápido) y a 15,18 µs de ejecutar los tres AFND por separado: el AFND debe mantener y expandir un conjunto de estados activos por símbolo, mientras que el AFD hace una sola consulta a δ. La minimización no cambia el tiempo por sesión (0,66 µs antes, 0,66 µs después, pues ambos hacen una consulta por símbolo), pero reduce la tabla de 41 a 9 estados. En el camino AFND → AFD el número de estados creció de 10 a 41 por la construcción de subconjuntos y volvió a 9 tras minimizar. Los tiempos absolutos dependen del equipo y de Python; lo relevante es la proporción.

8.3 Análisis de errores

Con 72 sesiones anómalas en validación y 11 en prueba, el monitor obtuvo cero falsos positivos y cero falsos negativos en ambos archivos, incluidas las 166 sesiones casi anómalas de validación (0 falsos positivos). Este resultado confirma que los autómatas reconocen exactamente los lenguajes definidos, pero hay que leerlo con cuidado: las etiquetas provienen de esas mismas definiciones, de modo que un 1,00 no demuestra que los patrones describan bien todos los incumplimientos reales de una biblioteca. Para mostrar que las métricas sí detectan un modelado incorrecto, se evaluaron dos variantes erróneas sobre el mismo archivo de validación:

Tabla 18. Prueba de sensibilidad con modelados erróneos de los patrones (validación).

Variante	VP	FP	FN	Precisión	Exhaustividad
P1 modelado como «contiene NN» (en vez de NNN)	72	88	0	0,45	1,00
P3 modelado como «contiene D» (sin exigir ausencia de P antes)	72	309	0	0,19	1,00

Ambas variantes detectan todas las anomalías (exhaustividad 1,00) pero marcan como anómalas muchas sesiones normales, lo que hunde la precisión. Las sesiones casi anómalas y las devoluciones legítimas son justamente las que permiten distinguir un modelo correcto de uno impreciso.

9. Límites del modelo

Regla no regular. «Nunca puede haber más devoluciones que préstamos», con cualquier cantidad de libros, es el lenguaje $L = \{w \in \Sigma^* \mid$ en todo prefijo de w, el número de D no supera al de P$\}$. No es regular. Por el lema de bombeo [2]: si L fuera regular con longitud de bombeo p, la cadena $w = P^p D^p \in L$ se descompone como xyz con $|xy| \leq p$ e $y = P^k$ ($k \geq 1$); al bombear con i = 0 se obtiene $P^{p-k} D^p \notin L$, una contradicción. Un autómata finito no puede contar hasta un número arbitrario porque solo recuerda una cantidad finita de información.

Adaptación. Si el máximo de libros prestados a la vez es fijo (por ejemplo, 3), la regla sí es regular y basta un AFD de 5 estados: c0, c1, c2 y c3 cuentan los préstamos pendientes y err es el estado de anomalía (absorbente).

Tabla 19. AFD de la regla con tope de 3 libros (R, N, M y G dejan el estado igual).

Estado	P	D	R, N, M, G
→ c0	c1	err	c0
c1	c2	c0	c1
c2	c3	c1	c2
c3	err (excede el tope)	c2	c3
err *	err	err	err

El patrón P3 detecta el error de devolver un libro sin tener préstamos previos, pero para aplicar esta regla en cualquier momento (asegurando que las devoluciones nunca superen los préstamos) necesitaríamos llevar un conteo. Aunque podríamos adaptarlo fijando un límite máximo de préstamos (lo que sumaría c + 2 estados al modelo), decidimos dejarlo como una mejora futura. Implementarlo sin límites requeriría un autómata con pila, un tema de unidades más avanzadas.

Por otro lado, dejamos fuera del modelo las reglas que dependen del tiempo (como calcular la multa por días de retraso). Nuestro autómata solo entiende el orden en que ocurren los eventos, no el tiempo real que pasa entre ellos; la fecha solo nos sirve para ordenarlos. Por lo tanto, el sistema de la biblioteca es el que calcula el retraso y registra la multa (evento M), mientras que nuestro monitor simplemente vigila que la secuencia de eventos alrededor de esa multa tenga sentido lógico (como en el patrón P2).

10. Conclusiones

- La vigilancia de secuencias anómalas, como renovaciones abusivas o préstamos tras multas, es altamente efectiva mediante lenguajes regulares. Los autómatas finitos logran detectar estas infracciones con precisión evaluando únicamente el orden de los eventos.
- Se demostró que los autómatas no deterministas (AFND) son ideales y flexibles para modelar los patrones, mientras que el autómata determinista (AFD) mínimo es óptimo para la fase de ejecución. Este último procesa todas las anomalías simultáneamente en una sola lectura.
- El proceso de minimización resulta indispensable, ya que reduce drásticamente el tamaño del autómata, pasando de 41 a solo 9 estados. Asimismo, someter el modelo a pruebas con historiales "casi anómalos" es el mejor método para validar su exactitud.

Referencias y declaración de uso de IA

[1] J. E. Hopcroft, R. Motwani y J. D. Ullman, Introduction to Automata Theory, Languages, and Computation, 3.ª ed. Boston, MA, EE. UU.: Pearson/Addison-Wesley, 2006.

[2] M. Sipser, Introduction to the Theory of Computation, 3.ª ed. Boston, MA, EE. UU.: Cengage Learning, 2013.

[3] S. H. Rodger y T. W. Finley, JFLAP: An Interactive Formal Languages and Automata Package. Sudbury, MA, EE. UU.: Jones & Bartlett, 2006.

[4] Seattle Public Library, «Checkouts by Title (Physical Items)», data.seattle.gov. [En línea]. Disponible en: https://cos-data.seattle.gov/d/5src-czff (consultado el 28 de octubre de 2026).

[5] Seattle Public Library, «Checkouts by Title», data.seattle.gov. [En línea]. Disponible en: https://cos-data.seattle.gov/d/tmmm-ytt6 (consultado el 28 de octubre de 2026).

[6] Universidad Nacional de Loja, «Evaluación sumativa 1 · Unidad 1. Proyecto grupal: análisis de secuencias de eventos con autómatas finitos. Lineamientos y plantillas», Teoría de Autómatas y Computabilidad Avanzada, Loja, Ecuador, 2026.

Declaración de uso de IA. Herramienta: Claude (Anthropic). Propósito y partes asistidas: (i) apoyo en el diseño y la escritura del generador de datos, de los módulos de Python (`automata.py`, `monitor.py`, `construir.py`) y de la verificación de equivalencia; (ii) cálculo de las tablas, las métricas y los diagramas de este informe ejecutando ese código; (iii) redacción de este informe a partir de la guía de la evaluación y de la propuesta del equipo (escenario, alfabeto, patrones y regla no regular).